FACULTY OF ENGINEERING AND TECHNOLOGY

GROUP 18

Course Code: CEF440

Course Title: Internet Programming (J2EE) and Mobile Programming

SN	**NAMES**	**MATRICULE**
1	NKONGHO ELIZABETH AGBOR-MANYO	FE23A122
2	TANDO CALVIN NKUH	FE23A159
3	DIVINE JEMBONG TABI NDIP	FE23A035
4	LENA KEZIAH ABANDA GHA	FE23A081
5	MENYE EWODO GERMAINE LAETICIA	FE23A092

Course Instructor: Dr.Nkemeni Valery

Table of Contents

Introduction to Mobile Development 3

1. Major Types of Mobile Applications 3

Main Differences and Comparison 4

2. Mobile App Programming Languages 4

3. Mobile App Development Frameworks 5

4. Mobile Application Architecture and Design Patterns 6

5. Requirement Engineering Process 8

- Feasibility Study 9
- Elicitation (Gathering Needs) 9
- Analysis 9
- Specification (Documentation) 9
- Validation 9
- Management 9

6.1 Identify Stakeholders 9

6.2 Gather Requirements 10

6.3 Analyse Requirements 10

6.4 Document Requirements 10

6.5 Validate Requirements 10

6.6 Manage Requirements Changes 10

6.7 Iteration 10

6. Mobile App Cost Estimation in Cameroon 11

What these categories mean 11

Timeline 11

Factors That Affect Cost 11

Where the Money Goes 11

Key Considerations 12

Conclusion 13

Introduction to Mobile Development

Mobile Application Development is the set of processes and procedures used to write software for small, wireless computing devices like smartphones and tablets. Even though it is based on traditional software engineering, mobile development is unique because developers have to deal with specific hardware constraints like battery life, smaller screen sizes, and different operating systems.

When starting a project, a developer must choose the right "path" based on what the app needs to do. This report looks at the technical side of these choices, focusing on how apps are built, the languages used to write them, and the frameworks that help finish projects more efficiently.

1. Major Types of Mobile Applications

In the mobile world, there are three main ways to build an app. Each one has a different architecture, which basically means how the app is structured and how it communicates with the phone's hardware.

1. <u>Native Applications</u>

Native apps are developed specifically for one mobile operating system. For example, if building for Android, Google's tools are used; for iOS, Apple's tools are required.

- Performance: They are the fastest because they are native to the device and don't need any translation layers.
- Access: They have the best access to things like the Camera, GPS, and Contact list.
- Downside: The main problem is that you have to write two different versions of the same app if you want it on both the Play Store and the App Store.

2. <u>Web Applications</u>

These aren't actually apps that are downloaded. They are really just high-quality, responsive websites designed to look like an app. They are opened using a browser like Chrome or Safari.

- Tools: They are built using standard web technologies like HTML5, CSS, and JavaScript.
- Advantage: You only build it once, and it works on every phone immediately.
- Disadvantage: They usually need a constant internet connection and can't use most of the phone's hardware features like the fingerprint scanner.

3. Hybrid Applications

Hybrid apps are a mix of both. They are built with web code but "wrapped" inside a native container. This lets them be installed from an app store just like a native app.

- Efficiency: They allow for the use of one codebase for both platforms, which is great for small teams or students.
- Performance: They are generally a bit slower than native apps, but they are much faster to build.

Main Differences and Comparison

Parameters	Web	Native	Hybrid
Performance	Slower; less intuitive	Excellent; quick and engaging	Moderate; functions according to UI
Cost & Time	Quickest and least expensive	Costly and time-consuming	Lower than native; easier maintenance
Device Access	Limited browser-reliant access	Direct, end-to-end access	Access via plugins or API calls
Deployment	Immediate (via URL)	App Store / Play Store	App Store / Play Store

2. Mobile App Programming Languages

A programming language is the set of instructions given to the computer. In mobile development, the language picked depends on which type of app was decided upon in the previous section.

3.1 Native Languages

- Kotlin: This is the modern, official language for Android. It's much easier to read than the older Java and helps prevent common crashes.
- Swift: This is the official language for iOS. Apple made it to be fast, safe, and modern.

- Java: The original language for Android. While Kotlin is more popular now, many big professional apps are still written in Java.
- Objective-C: The old language for Apple apps. It's a bit hard to learn, but some older projects still use it.

3.2 Cross-Platform Languages

These are very popular right now because they let one developer write code that works on both Android and iPhone at the same time.

- Dart: Created by Google. It is used with the Flutter framework and is very good at making smooth user interfaces.
- JavaScript: Most web developers already know this. It is used with React Native to build apps that feel like native apps.

3.3 Other Development Languages

- C++: Sometimes used for mobile games (like 3D shooters) because it's extremely fast.
- HTML5/CSS3: Even though these are technically markup languages, they are essential for the layout and design of web and hybrid apps.

3. Mobile App Development Frameworks

A Framework is like a toolbox. Instead of writing every single thing from scratch, developers use a framework that gives them pre-made components like buttons, lists, and navigation menus.

Framework	Language	Created By	Why Use It?
Flutter	Dart	Google	Great for making apps that look exactly the same on every device.
React Native	JavaScript	Meta	Perfect if you already know web development. Very popular in the industry.
Ionic	HTML/CSS/JS	Independent	Best for making simple apps or prototypes very quickly.
Xamarin	C#	Microsoft	Good for developers who are already used to the Microsoft/Windows ecosystem.

4. Mobile Application Architecture and Design Patterns

An architecture pattern is a set of guidelines and best practices that describe how to structure and organize code, data, and components. It defines the roles and responsibilities of each element, the communication and interaction between them, and the overall design goals and principles. An architecture pattern can help you achieve consistency, modularity, scalability, and performance in your software system. It can also help you avoid common pitfalls and errors, such as tight coupling, code duplication, and poor testability.

5.1 Why use an architecture pattern for mobile development?

Mobile development poses some unique challenges that require careful design decisions. For example, you have to consider the limited resources and battery life of mobile devices, the varying screen sizes and orientations, the network connectivity and latency, the user expectations and preferences, and the security and privacy issues. Using an architecture pattern can help you address these challenges by providing a clear and proven way of organizing your code and data.

Types of Architectural Design Patterns:

There are very few architectural design patterns available for mobile development.

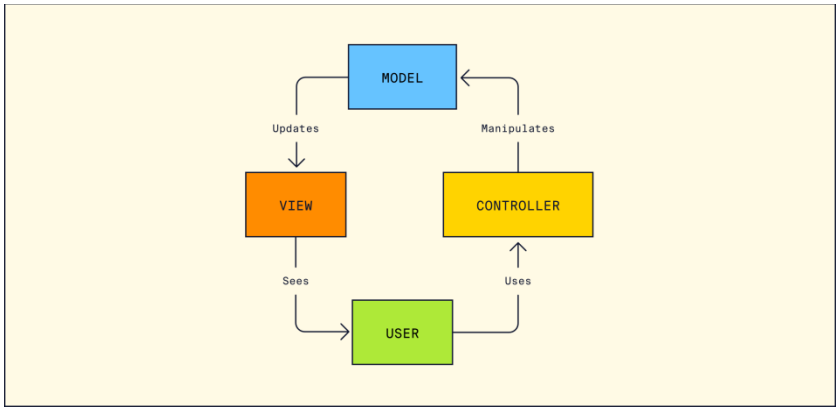
5.2 Model View Controller (MVC) Architecture

MVC is a design model that separates an application into three interacting parts: Model, View, and Controller. This separation allows for better code design and modularization.

- **Model**: Represents application data and business logic.
  Key responsibilities of the Model:
  - Stores and manages application data
  - Defines business rules and logic
  - Handles data validation

- **View**: Displays data to the user.
  Key responsibilities of the View:
  - Displays data to users
  - Handles user interface elements
  - Manages layout and styling
- **Controller**: Processes user input and controls data flow between Model and View.
  Key responsibilities of the Controllers:
  - Processes user input
  - Updates the model base on the user action
  - Handle Application flow and navigation

The figure below gives us a view of what this architecture looks like



5.3 Model View Presenter (MVP) Architecture

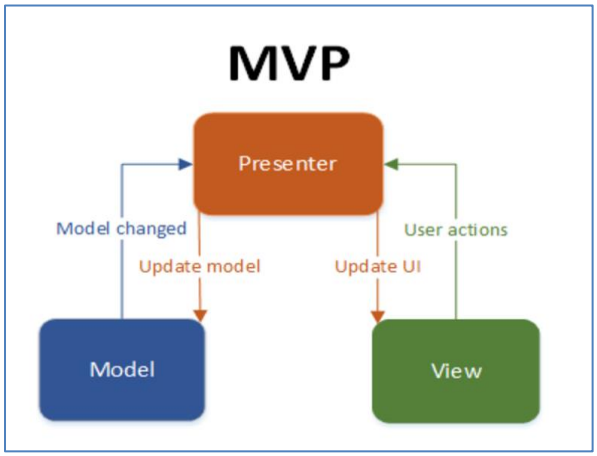
MVP is a new architecture that separates an application into three parts: Model, View, and Presenter. This is similar to MVC but puts more responsibility on the Teacher to manage the interaction between Model and View.

- Model: Manages data and business logic.
- View: Represents the user interface.
- Presenter: Acts as an intermediary processing user input and updating the View and Model.

Key responsibilities of the Presenter:

- Handles all the presentation logic.
- Receives input from the View, processes it, and then updates the View.

The figure below gives us a view of what this architecture looks like



5.4 Model-View-View Model (MVVM) Architecture

MVVM is a design model widely used in mobile development, especially in frameworks like Android's Jetpack. Its purpose is to separate the application into three parts: Model, View, and ViewModel.

• **Model**: Represents data and business logic.

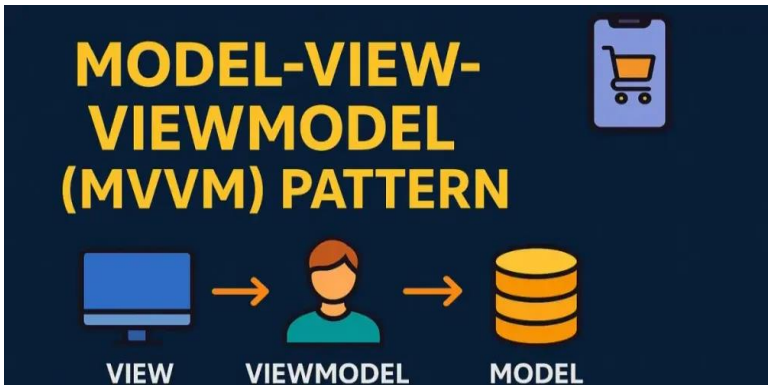
• **View**: Represents the user interface.

• **ViewModel**: Acts as an interface between the Model and the View, which contains the reference logic.

Key responsibilities of the ViewModel:

- Exposing data for the view
- Handling Commands and User Interaction

The figure below gives us a view of what this architecture looks like



5. Requirement Engineering Process

Requirements engineering is the process of clearly defining what a software system should do before development starts. It helps avoid confusion, reduces wasted effort, and ensures the final product actually solves the user's problem.

The process usually follows these steps:

- **Feasibility Study**

This step checks whether the project is realistic in terms of cost, available technology, and time. For example, building a school management app may sound simple, but the team must confirm that they have the skills, tools, and budget to complete it.

- **Elicitation (Gathering Needs)**

This involves talking to users, clients, and stakeholders to understand what they need. For instance, teachers may want attendance tracking, while parents may want progress reports and notifications.

- **Analysis**

Here, the requirements are examined to ensure they are clear, consistent, and achievable.Conflicts are also resolved. For example, if parents want daily updates but teachers can only provide weekly ones, a compromise must be reached.

- **Specification (Documentation)**

The requirements are written in a structured document, often called a Software Requirements Specification (SRS).

Example:
"The system should allow students to log in using a unique ID and password."

- **Validation**

This step ensures the documented requirements match what stakeholders actually want.This can involve showing mockups or simple prototypes to get feedback before development begins.

- **Management**

Requirements may change over time, so this step involves tracking and managing updates.

For example, if the school later wants online fee payment, it is added and managed properly.

Before starting the development phase, a structured process is used to define the exact needs of the users and stakeholders.

<u>6.1 Identify Stakeholders</u>

Identify everyone involved, including end-users, business owners, developers, and designers.

6.2 Gather Requirements

Use techniques like interviews for deep insights, surveys for broad feedback, workshops for brainstorming, and observations of users in their natural environment.

6.3 Analyse Requirements

Evaluate gathered information for clarity to avoid ambiguity, completeness of features, consistency between different needs, and technical feasibility within project constraints.

6.4 Document Requirements

Store requirements in structured formats such as Use Cases for system interactions, User Stories for concise user needs, or formal Requirement Specifications for detailed functional descriptions.

6.5 Validate Requirements

Confirm accuracy with stakeholders through document reviews and prototyping to provide a visual representation early in the process.

6.6 Manage Requirements Changes

Use a formal change control process to evaluate proposed updates and ensure documentation stays aligned throughout the project.

6.7 Iteration

Requirement engineering is continuous; developers must refine requirements based on new insights and ongoing user feedback.

In summary, requirements engineering ensures the system is useful, realistic, and aligned with user needs.

6. Mobile App Cost Estimation in Cameroon

The cost of developing a mobile app in Cameroon depends on complexity, features, and design quality.

Our findings from local developer pricing trends and small agency estimates in Cameroon suggest that costs could range from:

- **Basic apps:** around **150,000 to 300,000 XAF**
- **Standard apps:** around **300,000 to 700,000 XAF**
- **Complex apps: 700,000 XAF and above**

What these categories mean

- **Basic App**
  Simple interface with essential features, usually built for one platform.
  Example: a school app with announcements and attendance.
- **Standard App**
  More refined design with interactive features like dashboards and user accounts.
- **Complex App**
  Advanced features such as payments, GPS tracking, real-time updates, or multi-platform support.

Timeline

Our findings suggest that simple apps could take about **one to two months**, while more complex apps may take longer depending on features.

Factors That Affect Cost

- Number of features (login, payments, notifications)
- Platform choice (single or multiple platforms)
- Design quality
- Maintenance and updates after launch

Where the Money Goes

Our findings from small development teams and freelance projects suggest that most of the cost is spent in these key areas:

- **Planning and Requirements**
  Understanding the problem, meeting with stakeholders, and defining what the system should do.
- **Design (UI/UX)**
  Creating the look and feel of the app, including layouts, screens, and user experience.
- **Development**
  This includes building both the visible part of the app (what users interact with) and the background systems (databases, servers, and logic).
- **Testing and Deployment**
  Checking for errors, improving performance, and preparing the app for release on platforms like the Play Store.
- **Maintenance**
  Fixing bugs, updating features, and keeping the system running after launch.

Key Considerations

- **Design quality matters:** A better design improves user experience but increases cost.
- **Advanced features increase cost:** Payments, GPS, and real-time systems require more work.
- **Hidden costs exist:** Hosting, domains, and app store fees are often overlooked.
- **Team choice affects pricing:** Freelancers may charge less, while agencies offer more structure.
- **Planning for growth is important:** Building for scalability early can save money later.

Conclusion

In conclusion, developing a successful mobile application requires a strategic balance between technical performance, architectural integrity, and financial planning. By understanding the core differences between native, web, and hybrid development, teams can select the path that best aligns with their goals and budget. Furthermore, the implementation of robust architectural patterns like MVC or MVVM, combined with a disciplined requirement engineering process, ensures that the application remains scalable and maintainable. Ultimately, a well-estimated cost structure that accounts for everything from initial complexity to post-launch maintenance provides the necessary roadmap for turning a conceptual mobile project into a high-quality, functional reality.

References

- *Amorserv*
  Mobile app cost in Africa and development insights
- *Apptunix*
  Mobile app cost estimation and pricing factors
- *iSolve Mobility*
  App development cost and process explanation
- *GeeksforGeeks*
  Requirements engineering and software concepts
- *W3Schools*
  Web technologies and programming basics
- *Flutter Official Documentation*
  Cross-platform mobile development
- *React Native Documentation*
  JavaScript-based mobile app development